Universidade Federal do Cariri - UFCA

Proposta Técnica de Projeto

Programação Orientada a Objetos

Equipe:

Sebastião Sousa Soares
Espedito Ramom Mascena Ricarto
Sabrina Alencar Soares

Juazeiro do Norte - CE
Janeiro de 2026

Sumário

1 Identificação da Equipe **2**

2 Visão Geral do Projeto **2**
2.1 Descrição do Domínio 2
2.2 Justificativa da Complexidade e Escolha do Paradigma 2

3 Arquitetura e Modelagem **3**
3.1 Diagrama de Classes (UML) 3
3.2 Especificação de Hierarquias 3

4 Padrões e Princípios **4**
4.1 Padrões de Projeto (Design Patterns) 5
4.2 Princípios SOLID 5

1 Identificação da Equipe

- **Sebastião Sousa Soares** (`https://github.com/sebastiaosoares`)
- **Espedito Ramom Mascena Ricarto** (`https://github.com/ramomricarto`)
- **Sabrina Alencar Soares** (`https://github.com/sabrinaalencaar`)

2 Visão Geral do Projeto

2.1 Descrição do Domínio

O projeto, denominado de **Trocadu**, consiste em um jogo digital do estilo *Party Game*, feito para ser jogado em grupo usando apenas um celular (passando o aparelho de mão em mão). O objetivo do sistema é controlar automaticamente as regras de uma partida de adivinhação, para que os jogadores não precisem marcar pontos ou cronometrar o tempo manualmente.

A dinâmica funciona com duplas temporárias, onde os jogadores trocam de papel a cada turno:

- **Descritor:** Vê a palavra na tela e tenta descrevê-la (com mímicas ou dicas) sem falar a palavra exata.
- **Adivinhador:** Tenta acertar a palavra antes que o tempo acabe.

O jogo organiza um rodízio para garantir que todo mundo jogue com todo mundo. Cada dupla joga duas vezes (uma "Ida"e uma "Volta"), para que os dois tenham a chance de adivinhar e de descrever.

O sistema também gerencia recursos como o botão de "Pular"palavra e permite que quem criou a partida configure as regras, como o tempo de cada rodada e a lista de palavras que será usada.

2.2 Justificativa da Complexidade e Escolha do Paradigma

Escolhemos usar o paradigma **Orientado a Objetos (POO)** porque tentar fazer esse jogo de forma procedural (apenas com funções e variáveis globais) deixaria o código desorganizado e difícil de manter. Precisamos controlar o estado de vários objetos ao mesmo tempo (o tempo correndo, os pontos da equipe, qual palavra está ativa), e o **encapsulamento** da POO protege esses dados de erros.

Além disso, a aplicação dos princípios **SOLID** foi fundamental para organizar essa complexidade. Ao separar as responsabilidades (como o controle de tempo isolado na classe `Turno`) e utilizar interfaces para as fontes de palavras, conseguimos evitar que o código se tornasse rígido e difícil de corrigir. O uso de Objetos bem definidos também simplifica a lógica do rodízio de duplas ("Todos contra Todos"), permitindo manipular as equipes de forma muito mais segura e clara do que se tentássemos controlar esses relacionamentos apenas com arrays soltos em um código procedural.

3 Arquitetura e Modelagem

3.1 Diagrama de Classes (UML)

Abaixo apresentamos o diagrama de classes principal do sistema, evidenciando as relações entre as entidades de domínio.

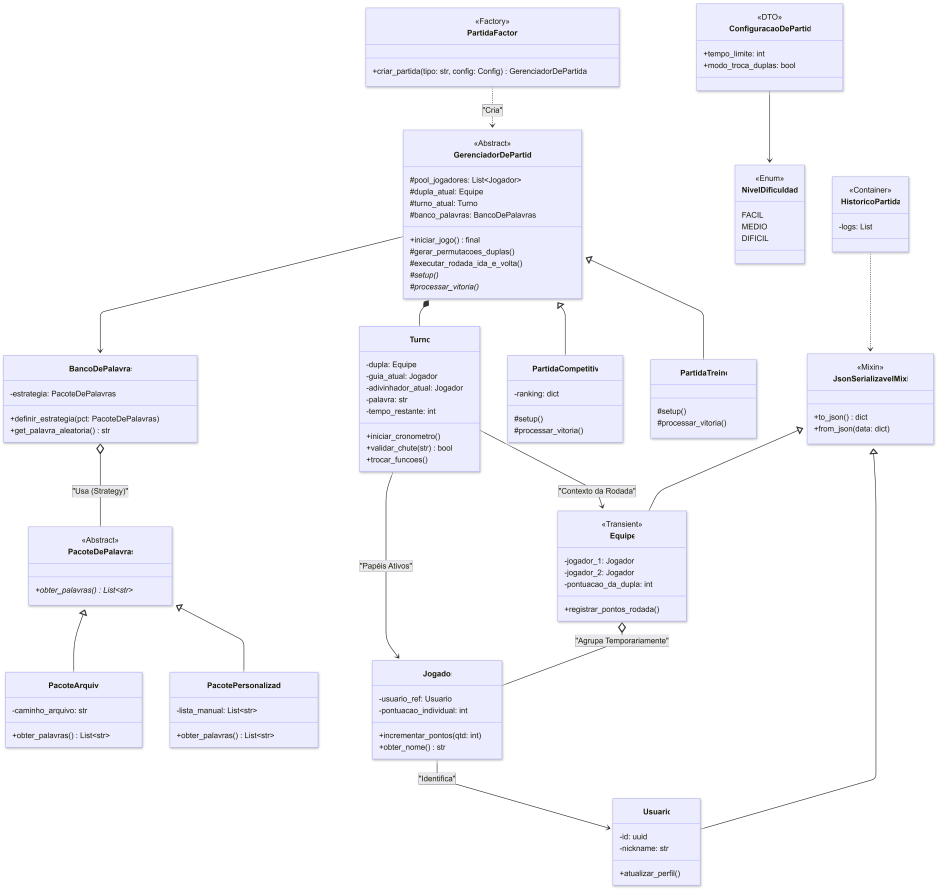

Figura 1: Diagrama de Classes do Domínio

3.2 Especificação de Hierarquias

O projeto contempla duas hierarquias principais e o uso de herança múltipla via Mixins, atendendo aos requisitos de abstração conceitual e variação de comportamento.

1. **Hierarquia de Abstração Conceitual (Fonte de Dados)**

   Esta hierarquia foca no contrato de obtenção de dados, isolando a origem das palavras da lógica do jogo.

   Classe Abstrata (Pai): `PacoteDePalavras`

   Define o contrato `obter_palavras()`, obrigando que qualquer fonte de dados devolva uma lista de strings.

   Subclasses (Filhas):

   - `PacoteArquivo`: Implementa a leitura de palavras a partir de fontes estáticas (arquivos JSON locais).

- `PacotePersonalizado`: Implementa a busca de palavras criadas pelo próprio usuário (em memória ou banco de dados).

Justificativa: Esta hierarquia representa uma abstração conceitual do domínio "Fonte de Palavras". O polimorfismo permite que o `BancoDePalavras` troque a fonte de dados (Arquivo ou Personalizado) sem alterar seu código interno, facilitando a expansão para novas fontes (como APIs) no futuro.

2. **Hierarquia de Variação de Comportamento (Regras do Jogo)**

   Esta hierarquia altera o funcionamento do fluxo do jogo, modificando como a partida é processada e finalizada.

   Classe Abstrata (Pai): `GerenciadorDePartida`
   Implementa o esqueleto do jogo (método `iniciar_jogo`) usando o padrão Template Method. Define métodos abstratos (`setup`, `processar_vitoria`) que devem ser implementados pelas filhas.

   Subclasses (Filhas):

   - `PartidaCompetitiva`: Implementa regras estritas de vitória, validação de ranking e persistência de estatísticas oficiais.
   - `PartidaTreino`: Sobrescreve as regras para permitir "loop infinito"ou ignorar pontuações no ranking, focando apenas na mecânica de treino.

   Justificativa: Esta hierarquia atende ao requisito de polimorfismo essencial. O código principal depende apenas da abstração `GerenciadorDePartida`, mas o comportamento real (se vale pontos ou não, se o jogo acaba ou repete) varia completamente dependendo da instância concreta instanciada.

3. **Herança Múltipla e Mixins (Funcionalidade Transversal)**

   Para evitar acoplamento desnecessário, utilizamos herança múltipla através de Mixins para funcionalidades auxiliares.

   Mixin: `JsonSerializavelMixin`
   Fornece métodos genéricos `to_json()` e `from_json()` para transformar objetos em dicionários.

   Classes que herdam:

   - `Usuario`
   - `Equipe`

   Justificativa: Em vez de criar uma superclasse genérica para todos os objetos do sistema, usamos um Mixin para injetar a capacidade de serialização apenas nas entidades que precisam ser salvas ou transmitidas, respeitando o princípio de composição sobre herança rígida.

4 Padrões e Princípios

Para resolver problemas recorrentes de arquitetura e garantir a escalabilidade e a manutenibilidade do código, foram adotadas soluções consagradas da engenharia de software, conforme detalhado a seguir.

4.1 Padrões de Projeto (Design Patterns)

Foram aplicados três padrões do catálogo *Gang of Four* (GoF) para estruturar as responsabilidades do sistema:

Template Method (Comportamental): Aplicado na classe abstrata `GerenciadorDePartida`. Esse padrão define o esqueleto do algoritmo principal do jogo por meio do método `iniciar_jogo()`, estabelecendo uma ordem fixa de execução das etapas — como configuração inicial, loop de turnos e verificação de vitória. As subclasses `PartidaCompetitiva` e `PartidaTreino` implementam apenas os métodos abstratos específicos (`setup` e `processar_vitoria`), permitindo a variação do comportamento sem alterar a estrutura do fluxo principal.

Factory Method (Criacional): Representado pela classe `PartidaFactory`, responsável por encapsular a lógica de criação das partidas. Dessa forma, o cliente do sistema não precisa conhecer nem instanciar diretamente classes concretas. O cliente apenas solicita uma partida à fábrica, fornecendo uma `ConfiguracaoDePartida`, e a fábrica decide qual objeto retornar. Essa abordagem desacopla a criação do objeto de seu uso, tornando o sistema mais flexível.

Strategy (Comportamental): Identificado na relação entre `BancoDePalavras` e a hierarquia `PacoteDePalavras`. O `BancoDePalavras` atua como o contexto que necessita de uma estratégia para obtenção de dados. As classes `PacoteArquivo` e `PacotePersonalizado` representam estratégias concretas intercambiáveis. Isso permite que o algoritmo de obtenção de palavras seja alterado em tempo de execução (alternando entre arquivo JSON ou memória) sem modificar a classe principal.

4.2 Princípios SOLID

A aplicação dos princípios SOLID guiou a modelagem e a refatoração, contribuindo para a redução do acoplamento e aumento da coesão.

S - Single Responsibility Principle (SRP): Garante que cada classe possua apenas uma razão para mudar. Um exemplo claro é a separação entre a classe `GerenciadorDePartida` (fluxo macro do jogo) e a classe `Turno` (lógica temporal e pontuação da rodada). Essa divisão evita a sobrecarga de responsabilidades e simplifica manutenções futuras.

O - Open/Closed Principle (OCP): O sistema está aberto para extensão, mas fechado para modificação. A abstração `PacoteDePalavras` possibilita a criação de novas fontes — como `PacoteAPI` — sem alterar a classe `BancoDePalavras`. O mesmo se aplica aos novos modos de jogo via extensão do `GerenciadorDePartida`.

L - Liskov Substitution Principle (LSP): As subclasses `PartidaCompetitiva` e `PartidaTreino` podem substituir a classe base `GerenciadorDePartida` em qualquer ponto do sistema sem quebrar o funcionamento. O contrato dos métodos abstratos assegura que o comportamento fundamental do jogo seja preservado.

I - Interface Segregation Principle (ISP): Aplicado através de Mixins, como o `JsonSerializavelMixin`, evitando superclasses genéricas. Apenas as classes que

realmente necessitam de serialização — como `Usuario` e `Equipe` — herdam esse comportamento, mantendo as demais classes com interfaces limpas.

D - Dependency Inversion Principle (DIP): Módulos de alto nível dependem de abstrações, não de implementações. A classe `BancoDePalavras` não depende da leitura de arquivos físicos (`PacoteArquivo`), mas sim da abstração `PacoteDePalavras`. Isso desacopla a regra de negócio dos detalhes técnicos de infraestrutura.